

Computer Organization and Architecture		
Lab #:	12	
Lab Title:	Implements a 32-bit 4-1 mux without using conditional operator	
Submitted by:		
Names		Registration #
Sumaira Safeer		FA22-BCE-019

Rubrics name & number					Marks	
					In-Lab	Post-Lab
Engineering Knowledge	R2: Use of Engineering Knowledge and follow Experiment Procedures: Ability to follow experimental procedures, control variables, and record procedural steps on lab report.					
Problem Analysis	R6: Experimental Data Analysis: Ability to interpret findings, compare them to values in the literature, identify weaknesses and limitations.					
Design	R8: Best Coding Standards: Ability to follow the coding standards and programming practices.					
Modern Tools Usage	R9: Understand Tools: Ability to describe and explain the principles behind and applicability of engineering tools.					
Individual and Teamwork	R12: Individual Work Contributions: Ability to carry out individual responsibilities.					
	R13: Management of Team Work: Ability to appreciate, understand and work with multidisciplinary team members.					
Rubrics #	R2	R6	R8	R9	R12	R13
In Lab						
Post- Lab						

Implements a 32-bit 4-1 mux without using Conditional Operator

Task: implements a 32-bit 4-1 mux without using conditional operator.

Objective:

The objective of this task is to design and implement a 32-bit 4:1 Multiplexer using combinational logic without employing any conditional (?:) operators. The primary goals are:

1. To understand the behavior of a 4:1 multiplexer, which selects one of the four 32-bit inputs based on a 2-bit selection signal.
2. To implement the multiplexer using bitwise operations, logical gates, or similar alternatives instead of relying on high-level conditional constructs.
3. To validate the correctness and efficiency of the design through testing and simulation.

Verilog Code:

```
module mux4to1_32bit (  
    input [31:0] in0,    // Input 0  
    input [31:0] in1,    // Input 1  
    input [31:0] in2,    // Input 2  
    input [31:0] in3,    // Input 3  
    input [1:0] sel,     // 2-bit Select Line  
    output reg [31:0] out // Output  
);  
// Combinational logic using case statement  
always @(*) begin  
    case (sel)  
        2'b00: out = in0;        // Select Input 0  
        2'b01: out = in1;        // Select Input 1  
        2'b10: out = in2;        // Select Input 2  
        2'b11: out = in3;        // Select Input 3  
        default: out = 32'b0;    // Default case  
    endcase  
end
```

```
end  
endmodule
```

Testbench:

```
module mux4to1_32bit (  
    input [31:0] in0, // Input 0  
    input [31:0] in1, // Input 1  
    input [31:0] in2, // Input 2  
    input [31:0] in3, // Input 3  
    input [1:0] sel, // 2-bit Select Line  
    output reg [31:0] out // Output  
);  
// Combinational logic using case statement  
always @(*) begin  
    case (sel)  
        2'b00: out = in0; // Select Input 0  
        2'b01: out = in1; // Select Input 1  
        2'b10: out = in2; // Select Input 2  
        2'b11: out = in3; // Select Input 3  
        default: out = 32'b0; // Default case  
    endcase  
end  
endmodule
```

Result:

